

- 8.1 设 $G_1 = \langle V_1, E_1 \rangle, G_2 = \langle V_2, E_2 \rangle$ 是两个简单图, 其中 $V_1 = V_2$. 假设 G_1 和 G_2 的输入是用邻接矩阵表示的. 换句话说, 如果 (v_i, v_j) 是图的边, 那么它的邻接矩阵 M 的第 i 行第 j 列的元素 $r_{ij} = 1$; 否则 $r_{ij} = 0, 1 \leq i, j \leq n$.
- (1) 设输入规模是图中的顶点数 n , 给出一个算法判定 G_1 是否为 G_2 的补图. 说明算法的设计思想, 并给出最坏情况下的时间复杂度.
- (2) 对于求解这个问题的所有算法, 给出一个尽可能紧的时间复杂度的下界, 并证明你的结果.

解: (1) 逐个比较 G_1 和 G_2 中的元素, 若 $\exists (i, j), G_1[i, j] = G_2[i, j]$ 则 G_1 不是 G_2 的补图, 否则 G_1 是 G_2 的补图. 由于 G_1, G_2 是无向图, 故只需比较上三角矩阵. $W(n) = \frac{n(n-1)}{2}$

(2) 对于求解该问题的任意算法 A , A 在最坏情况下的时间复杂度下界为 $\frac{n(n-1)}{2}$

证: 该问题等价于比较两个大小为 $n \times n$ 的 0-1 矩阵是否互补, 要求解该问题, 最坏情况下需进行至少 $\frac{n(n-1)}{2}$ 次比较以检查所有元素

$$\text{故 } W(n) = \frac{n(n-1)}{2}$$

8.2 对于给定的 $x \neq 0$, 求 n 次多项式 $P(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$ 的值.

(1) 设计一个在最坏情况下时间复杂度为 $\Theta(n)$ 的求值算法.

(2) 证明任何求值算法的时间复杂度都是 $\Omega(n)$

解: (1) 采用分治算法.

$$P(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$$

$$= P_{\text{even}}(x) + xP_{\text{odd}}(x)$$

① n 为偶数

$$P_{\text{even}}(x) = a_0 + a_2x + \dots + a_nx^{\frac{n}{2}}$$

$$P_{\text{odd}}(x) = a_1 + a_3x + \dots + a_{n-1}x^{\frac{n}{2}-1}$$

② n 为奇数

$$P_{\text{even}}(x) = a_0 + a_2x + \dots + a_{n-1}x^{\frac{n-1}{2}}$$

$$P_{\text{odd}}(x) = a_1 + a_3x + \dots + a_nx^{\frac{n-1}{2}}$$

$$\Rightarrow T(n) = 2T(\frac{n}{2}) + O(1)$$

$$\Rightarrow T(n) = \Theta(n)$$

(2) 证: 用求值算法 A .

$$P(x) \text{ 是 } n \text{ 次多项式} \Rightarrow a_n \neq 0$$

A 求 $P(x=x_0)$ 至少要做 $n-1+1=n$ 次运算 ($x_0 \neq 0$)

$$\Rightarrow A \text{ 的时间复杂度 } T(n) = \Omega(n)$$

(以乘法为关键运算).

8.2 (1) 迭代计算:

$$P_1(x) = a_n$$

$$P_2(x) = a_{n-1} + P_1(x) \cdot x$$

$$P_3(x) = a_{n-2} + P_2(x) \cdot x$$

$$\vdots$$

$$P_{n+1}(x) = a_0 + P_n(x) \cdot x$$

不难看出

$$P_{n+1}(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n = P(x)$$

(2) 证 多项式 $P(x)$ 有 $n+1$ 个数, 每个系数至少需要被处理 1 次. 如若不然, 假设某个求值算法 A 不对系数 a_i 进行处理, 考虑输入

$$\langle a_0, a_1, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_n \rangle \text{ 和 } \langle a_0, a_1, \dots, a_{i-1}, b_i, a_{i+1}, \dots, a_n \rangle$$

其中 $a_i \neq b_i$ 是不为 0 的数, 其他系数都相等. 那么这两个多项式对同一个 x 的求值一定不等, 但是算法 A 的输出是不加区别的. 因此算法 A 出错. 于是任何算法至少需要 $n+1$ 次运算, 即在最坏情况下时间复杂度为 $\Omega(n)$.

8.4 设 n 是 k 的倍数, 有 k 个排好序的数表 $L_1, L_2, \dots, L_k$, 每个数表都有 n/k 个数. 假设 n 个数彼此不等, 并且归并长为 m, n 的两个数表的时间代价是 $O(m+n)$.

(1) 使用顺序归并算法归并这 k 个数表, 在最坏情况下的时间复杂度是什么?

(2) 设计一个时间复杂度更低的归并算法, 说明算法的主要设计思想, 并分析你的算法在最坏情况下的时间复杂度.

(3) 对于以比较作为基本运算求解上述问题的算法类, 最坏情况下的时间复杂度的下界是什么? 证明你的结果.

解: (1) 最坏情况下归并长为 n, n 的两个数表的时间代价为 $n+n-1$

故算法在最坏情况下的时间复杂度为

$$\frac{2n}{k} + \frac{3n}{k} + \frac{4n}{k} + \dots + \frac{kn}{k} = \frac{n}{k} \cdot \frac{(k+2)(k-1)}{2} = \Theta(k \cdot n)$$

(2) 采用分治策略, 每次将数表均分为两部分, 分别归并两部分后再合并起来.

$$W(k, n) = 2W(\frac{k}{2}, \frac{n}{2}) + O(n)$$

$$\Rightarrow W(k, n) = \Theta(n \log k)$$

(3) 下界为 $\Theta(n \log k)$, 证明如下:

用决策树, 设 n 个数为 $a_1, a_2, \dots, a_n$. 构造规则如下:

设当前结点比较 i, j , 记为 (i, j)

① $i < j$,

若算法下一步比较 i', j' , 则将 (i, j) 的左子结点标记为 (i', j') ; 若算法结束, 则将 (i, j) 的左子结点标记为叶子结点.

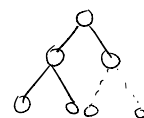
② $i > j$

若算法下一步比较 i', j' , 则将 (i, j) 的右子结点标记为 (i', j') ; 若算法结束, 则将 (i, j) 的右子结点标记为叶子结点.

每个叶子结点对应一种输入.

$$\text{所有可能的输入有 } \frac{n!}{(\frac{n}{k})! (\frac{n}{k})! \dots (\frac{n}{k})!} = \frac{n!}{((\frac{n}{k})!)^k} \text{ 种}$$

$$\Rightarrow \text{树深 } d \geq \lceil \log \frac{n!}{((\frac{n}{k})!)^k} \rceil$$



$$\geq \log n! - k \log (\frac{n}{k})!$$

$$= \Theta(\log n! - k \cdot \frac{n}{k} \log \frac{n}{k})$$

$$= \Theta(n \log n - n \log n + n \log k)$$

$$= \Theta(n \log k)$$

$$\Rightarrow \text{下界为 } \Theta(n \log k)$$

8.7 设 A 是 n 个不等的整数按照递增次序排列的数组, 已知存在 $i \in \{1, 2, \dots, n\}$ 使得 $A[i] = i$, 问怎样找到 i ?

(1) 设计一个算法求解上述问题, 给出算法的伪码描述并分析算法在最坏情况下的时间复杂度。

(2) 证明任何求解上述问题的算法至少需要做 $\Omega(\log n)$ 次比较。

解: (1) 采用二分查找。

BinarySearch(A, n)

$l \leftarrow 1$

$r \leftarrow n$

while $l \leq r$ do

$mid \leftarrow \frac{l+r}{2}$

if $A[mid] = mid$ then

return mid

if $A[mid] < mid$ then

$l \leftarrow mid$

else

$r \leftarrow mid$

$W(n) = O(\log n)$

(2) 证: 构造决策树, 规则如下:

每个结点表示一次比较, 设当前比较 i 与 $A[i]$,

将该结点记为 i 。

① $i \neq A[i]$, 若算法下一步比较 j , 则将 i 的左子结点标记为 j ;

② $i = A[i]$, 算法结束, 将 i 的右子结点标记为叶子结点。

决策树有 n 个叶子结点。

$\Rightarrow W(n) = \Omega(\log n) = \Omega(\log n)$

8.10 证明任何从 n 个数中选第 k 小的数的算法, 如果以比较作为基本运算, 那么它至少要做 $n + \min\{k, n-k+1\} - 2$ 次比较。

证: 构造最坏输入, 规则如下:

定义三种状态: N : 未经赋值

L : 赋值大于 k -value

S : 赋值小于 k -value

设算法在某一步比较 x, y

① $x: N, y: N \Rightarrow x: S, y: L$

② x, y 一个为 N 一个为 $L/S \Rightarrow$ 对 N 的数赋值, 使其变为 S/L

③ x, y 一个为 L 一个为 $S \Rightarrow$ 不变

④ x, y 均为 $L/S \Rightarrow$ 将 y 重新赋值, 使之变为 S/L 。

若算法在某一步时已还存在 $\min\{k, n-k+1\} - 1$ 个 L/S ,

则将未赋值的数全部赋值, 使之变为 S/L 。

将最后一个数赋值为 k -value。

冗余比较次数至少为 $\min\{k, n-k+1\} - 1$ (①)

必须由比较次数为 $n-1$ 。

故算法至少要做 $n + \min\{k, n-k+1\} - 2$ 次比较。

■

8.10 证 设第 k 小的数为 t , 首先定义决定性的比较与非决定性的比较。决定性的比较就是建立了 x 与 t 关系的比较。例如:

$\exists y(x > y \text{ 且 } y \geq t)$, x 满足上述条件的第一次比较

$\exists y(x < y \text{ 且 } y \leq t)$, x 满足上述条件的第一次比较

当 $x > t, y < t$ 时, $x > y$ 的比较不是决定性的。为找到第 k 小的数 t , 必须要做 $n-1$ 次决定性的比较。

② 设 A 是找第 k 小的任意算法, 针对算法 A 如下构造输入。

1. 分配一个值给第 k 小的数 t 。

2. 如果 A 比较 x 与 y , 且 x 与 y 没有被赋值, 那么赋值 x, y 使得 $x > t, y < t$ 。

3. 如果 A 比较 x 与 y , 且 $x > t, y$ 没被赋值, 则赋值 y 使得 $y < t$ 。

4. 如果 A 比较 x 与 y , 且 $x < t, y$ 没被赋值, 则赋值 y 使得 $y > t$ 。

5. 令 $c = \min\{k-1, n-k\}$, 如果存在 c 个元素已得到小于 t 的值, 则对未赋值的全部分配大于 t 的值。

6. 如果存在 c 个元素已得到大于 t 的值, 则对未赋值的全部分配小于 t 的值。

7. 如果剩下 1 个元素则分配 t 给它。

③ 这样赋值的输入使得 A 在这个输入下所进行的上述比较都是非决定性的。这样的比较至少有 $c = \min\{k-1, n-k\}$ 个。因此总比较次数至少为

$$n-1+c = n-1+\min\{k-1, n-k\} = n+\min\{k, n-k+1\}-2$$